

✓ prob stmt 3: Multi-Source Retail Sales Data Integration and Analysis

Retail Analytics

```
packages_needed <- c("readr", "jsonlite", "readxl", "dplyr", "RSQLite", "DBI",
to_install <- setdiff(packages_needed, rownames(installed.packages()))
if (length(to_install) > 0) install.packages(to_install)
```

```
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE,
                      fig.width = 6, fig.height = 4)
options(scipen = 999, digits = 4)
```

```
library(readr)      # CSV import
library(jsonlite)    # JSON import
library(readxl)      # Excel import
library(dplyr)       # data manipulation
library(RSQLite)     # SQLite driver
library(DBI)         # database interface
library(writexl)     # (used upstream to prepare customers.xlsx)
library(knitr)       # kable() tables
```

```
transactions_raw <- read_csv("transactions.csv", show_col_types = FALSE)
products_raw     <- fromJSON("products.json")
customers_raw    <- read_excel("customers.xlsx")
```

✓ Inspect the datasets

```
dim(transactions_raw)
dim(products_raw)
dim(customers_raw)
```

```
541909 · 5
4070 · 3
4372 · 2
```

```
str(transactions_raw)
str(products_raw)
str(customers_raw)
```

```
spec_tbl_ [541,909 × 5] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ InvoiceNo  : chr [1:541909] "536365" "536365" "536365" "536365" ...
 $ StockCode  : chr [1:541909] "85123A" "71053" "84406B" "84029G" ...
 $ CustomerID : num [1:541909] 17850 17850 17850 17850 17850 ...
 $ Quantity   : num [1:541909] 6 6 8 6 6 2 6 6 6 32 ...
 $ InvoiceDate: POSIXct[1:541909], format: "2010-12-01 08:26:00" "2010-12-01 08:
- attr(*, "spec")=
 .. cols(
 ..   InvoiceNo = col_character(),
 ..   StockCode = col_character(),
 ..   CustomerID = col_double(),
 ..   Quantity = col_double(),
 ..   InvoiceDate = col_datetime(format = "")
 .. )
- attr(*, "problems")=<pointer: 0x55dbf609e0c0>
'data.frame': 4070 obs. of 3 variables:
 $ StockCode : chr "10002" "10080" "10120" "10123C" ...
 $ Description: chr "INFLATABLE POLITICAL GLOBE" "GROOVY CACTUS INFLATABLE" "DC
 $ UnitPrice : num 0.85 0.39 0.21 0.65 0 0.42 0.42 0.85 0.79 0 ...
tibble [4,372 × 2] (S3: tbl_df/tbl/data.frame)
 $ CustomerID: num [1:4372] 12346 12347 12348 12349 12350 ...
 $ Country : chr [1:4372] "United Kingdom" "Iceland" "Finland" "Italy" ...
```

Missing values per column:

```
colSums(is.na(transactions_raw))
colSums(is.na(products_raw))
colSums(is.na(customers_raw))
```

InvoiceNo:	0	StockCode:	0	CustomerID:	135080	Quantity:	0
InvoiceDate:	0						
StockCode:	0	Description:	112	UnitPrice:	0		
CustomerID:	0	Country:	0				

Duplicate rows:

```
sum(duplicated(transactions_raw))
sum(duplicated(products_raw))
sum(duplicated(customers_raw))
```

```
5429
0
0
```

Invalid quantities / prices:

```
sum(transactions_raw$Quantity <= 0, na.rm = TRUE)
sum(products_raw$UnitPrice <= 0, na.rm = TRUE)
```

```
10624
133
```

Interpretation. `transactions.csv` holds `r nrow(transactions_raw)` line items.

About `r`

`round(100*sum(is.na(transactions_raw$CustomerID))/nrow(transactions_raw), 1)%` of rows have no `CustomerID` (guest / unidentified checkouts) and `r sum(duplicated(transactions_raw))` rows are exact duplicates (double-scanned lines). `r sum(transactions_raw$Quantity<=0)` rows carry a zero or negative `Quantity` -- these correspond to order cancellations, whose `InvoiceNo` starts with the letter "C". In `products.json`, `r sum(products_raw$UnitPrice<=0)` stock codes carry a `UnitPrice` of 0 and `r sum(is.na(products_raw$Description))` have no description; inspecting them shows they are internal write-off / adjustment codes (e.g. "damages", "thrown away", "Unsaleable, destroyed.") rather than sellable products.

✓ Clean the data

Cleaning decisions taken (summary):

1. **Transactions -- duplicates:** exact duplicate rows are removed with `distinct()`; these are treated as double-entry errors, not repeat sales.
2. **Transactions -- invalid quantities:** rows with `Quantity <= 0` are cancellations/returns, not sales. They are set aside in a separate `cancelled_transactions` table for reference and excluded from the revenue-bearing dataset used in Tasks 2--4.
3. **Transactions -- missing `CustomerID`:** rather than dropping these rows (which would understate total company revenue), they are **kept** for revenue/product totals and only excluded later from customer-level analysis, where a customer identity is required.
4. **Products -- invalid/zero `UnitPrice` or missing `Description`:** these are internal adjustment/write-off codes, not sellable products, so they are removed from the product master before revenue can be computed against them.
5. **Customers -- duplicates/missing `Country`:** duplicate `CustomerID` rows are collapsed to one row each; rows with no `Country` are dropped since country-level analysis needs it.

```

transactions <- transactions_raw %>%
  distinct() %>%
  mutate(
    InvoiceDate = as.POSIXct(InvoiceDate, format = "%Y-%m-%d %H:%M:%S"),
    IsCancelled = grepl("^C", InvoiceNo)
  )

cancelled_transactions <- transactions %>% filter(Quantity <= 0)
transactions_clean <- transactions %>% filter(Quantity > 0)

cat("Duplicates removed:", nrow(transactions_raw) - nrow(transactions), "\n")
cat("Cancelled/invalid-quantity rows set aside:", nrow(cancelled_transactions), "\n")
cat("Transactions remaining for revenue analysis:", nrow(transactions_clean), "\n")

```

```

Duplicates removed: 5429
Cancelled/invalid-quantity rows set aside: 10513
Transactions remaining for revenue analysis: 525967

```

```

products <- products_raw %>% distinct(StockCode, .keep_all = TRUE)

products_clean <- products %>%
  filter(UnitPrice > 0) %>%
  mutate(Description = ifelse(is.na(Description), "Unknown Product", Description))

cat("Products dropped (invalid price / write-off codes):",
    nrow(products) - nrow(products_clean), "\n")
cat("Clean product master rows:", nrow(products_clean), "\n")

```

```

Products dropped (invalid price / write-off codes): 133
Clean product master rows: 3937

```

```

customers_clean <- customers_raw %>%
  distinct(CustomerID, .keep_all = TRUE) %>%
  filter(!is.na(Country))

cat("Duplicate customers removed:",
    nrow(customers_raw) - nrow(distinct(customers_raw, CustomerID, .keep_all = TRUE)), "\n")
cat("Customers with missing country dropped:",
    nrow(distinct(customers_raw, CustomerID, .keep_all = TRUE)) - nrow(customers_clean), "\n")
cat("Clean customer master rows:", nrow(customers_clean), "\n")

```

```

Duplicate customers removed: 0
Customers with missing country dropped: 0
Clean customer master rows: 4372

```

✓ Create the **Revenue** attribute

Revenue requires UnitPrice, which lives only in the product master, so the first (product) join is performed here to create it. inner_join() is used deliberately: a transaction line can

only carry a revenue figure if it maps to a valid, priced product.

```
transactions_priced <- transactions_clean %>%
  inner_join(products_clean, by = "StockCode") %>%
  mutate(Revenue = Quantity * UnitPrice)

cat("Transaction rows before product match:", nrow(transactions_clean), "\n")
cat("Transaction rows with a valid priced product (Revenue computable):",
    nrow(transactions_priced), "\n")
cat("Rows dropped (matched an invalid/write-off product code):",
    nrow(transactions_clean) - nrow(transactions_priced), "\n")

kable(head(transactions_priced, 5), caption = "Sample of transactions with Reve
```

```
Transaction rows before product match: 525967
Transaction rows with a valid priced product (Revenue computable): 525944
Rows dropped (matched an invalid/write-off product code): 23
```

Table: Sample of transactions with Revenue

InvoiceNo	StockCode	CustomerID	Quantity	InvoiceDate	IsCancelled
Description			UnitPrice	Revenue	
536365	85123A	17850	6	2010-12-01 08:26:00	FALSE
WHITE HANGING HEART T-LIGHT HOLDER			2.95	17.7	
536365	71053	17850	6	2010-12-01 08:26:00	FALSE
WHITE METAL LANTERN			3.75	22.5	

✓ Task 2: Integrate the Multiple Data Sources

The remaining join -- transactions (already priced) with the **customer** master -- is performed with `left_join()`. A `left_join()` (rather than `inner_join()`) is chosen deliberately: dropping unmatched rows would silently discard real, revenue-bearing sales simply because the buyer's identity is unknown. Keeping them lets Task 3 report *total* company revenue accurately, while customer-specific analyses later restrict to rows that do have a match.

```
retail_full <- transactions_priced %>%
  left_join(customers_clean, by = "CustomerID")

cat("Dimensions before customer join:", dim(transactions_priced), "\n")
cat("Dimensions after customer join :", dim(retail_full), "\n")
```

```
Dimensions before customer join: 525944 9
Dimensions after customer join : 525944 10
```

✓ Verify dimensions and unmatched records

```
guest_rows <- retail_full %>% filter(is.na(CustomerID))
id_no_country <- retail_full %>% filter(!is.na(CustomerID) & is.na(Country))

cat("Rows with no CustomerID at all (guest transactions):", nrow(guest_rows), "
cat("Rows with a CustomerID not found in the customer master:", nrow(id_no_cour
cat("Total unmatched (no Country available):", nrow(guest_rows) + nrow(id_no_cc
cat("Revenue from unmatched (unattributable) rows: £",
      format(round(sum(c(guest_rows$Revenue, id_no_country$Revenue)), 2), big.mar
```

```
Rows with no CustomerID at all (guest transactions): 133240
Rows with a CustomerID not found in the customer master: 0
Total unmatched (no Country available): 133240
Revenue from unmatched (unattributable) rows: £ 1,418,104
```

Join type justification. `inner_join()` was used for the *product* join in Task 1 because a row without a valid price cannot contribute a revenue figure -- there is nothing to keep. `left_join()` was used for the *customer* join because the row still carries valid, real revenue even without a known buyer; an `inner_join()` here would have quietly deleted `r nrow(guest_rows)` rows of genuine sales (about `r round(100*nrow(guest_rows)/nrow(retail_full),1)` % of the integrated dataset) and understated total revenue. The rows themselves are retained; they are simply excluded later wherever `Country` or `CustomerID` is a required grouping key.

\newpage

✓ Task 3: Sales and Customer Analysis

Total sales revenue

```
total_revenue <- sum(retail_full$Revenue)
cat("Total sales revenue: £", format(round(total_revenue, 2), big.mark = ","),
```

```
Total sales revenue: £ 11,108,511
```

✓ Top 5 products by revenue

```
top_products <- retail_full %>%
  group_by(StockCode, Description) %>%
  summarise(Revenue = sum(Revenue), UnitsSold = sum(Quantity), .groups = "drop")
```

```

arrange(desc(Revenue)) %>%
slice_head(n = 5)

kable(top_products, caption = "Top 5 products by revenue", digits = 2)

```

Table: Top 5 products by revenue

StockCode	Description	Revenue	UnitsSold
:-----	:-----	-----	-----
DOT	DOTCOM POSTAGE	316458	1708
22423	REGENCY CAKESTAND 3 TIER	176740	13862
23843	PAPER CRAFT , LITTLE BIRDIE	168470	80995
85123A	WHITE HANGING HEART T-LIGHT HOLDER	122838	41640
POST	POSTAGE	117000	6500

✓ Top 5 countries by revenue

Country is only known for rows that matched a customer record, so this analysis uses

```
retail_full %>% filter(!is.na(Country)).
```

```

top_countries <- retail_full %>%
  filter(!is.na(Country)) %>%
  group_by(Country) %>%
  summarise(Revenue = sum(Revenue), Orders = n_distinct(InvoiceNo), .groups = "
  arrange(desc(Revenue)) %>%
  slice_head(n = 5)

```

```
kable(top_countries, caption = "Top 5 countries by revenue", digits = 2)
```

Table: Top 5 countries by revenue

Country	Revenue	Orders
:-----	-----	-----
United Kingdom	7996411	16649
Netherlands	335522	95
EIRE	288215	260
Germany	235801	457
France	206020	389

✓ Top 5 customers by total purchase value

```

top_customers <- retail_full %>%
  filter(!is.na(CustomerID)) %>%
  group_by(CustomerID, Country) %>%

```

```
summarise(Revenue = sum(Revenue), Orders = n_distinct(InvoiceNo), .groups = "
arrange(desc(Revenue)) %>%
slice_head(n = 5)
```

```
kable(top_customers, caption = "Top 5 customers by total purchase value", digit
```

Table: Top 5 customers by total purchase value

CustomerID	Country	Revenue	Orders
18102	United Kingdom	403365	60
14646	Netherlands	330057	74
17450	United Kingdom	181054	46
16446	United Kingdom	168472	2
14911	EIRE	152203	201

✓ Customer value segmentation with `case_when()`

Each identified customer's total revenue is compared against the 25th, 75th and 95th percentiles of the customer revenue distribution, so thresholds adapt to this dataset rather than being picked arbitrarily.

```
customer_revenue <- retail_full %>%
  filter(!is.na(CustomerID)) %>%
  group_by(CustomerID) %>%
  summarise(TotalRevenue = sum(Revenue), .groups = "drop")

q <- quantile(customer_revenue$TotalRevenue, probs = c(0.25, 0.75, 0.95))
cat("Thresholds -> Low/Medium:", round(q[1],2),
    " Medium/High:", round(q[2],2),
    " High/Premium:", round(q[3],2), "\n")

customer_segments <- customer_revenue %>%
  mutate(Segment = case_when(
    TotalRevenue < q[1] ~ "Low Value",
    TotalRevenue < q[2] ~ "Medium Value",
    TotalRevenue < q[3] ~ "High Value",
    TRUE ~ "Premium"
  ))

segment_summary <- customer_segments %>%
  count(Segment, name = "Customers") %>%
  mutate(SharePct = round(100 * Customers / sum(Customers), 1)) %>%
  arrange(match(Segment, c("Low Value", "Medium Value", "High Value", "Premium")))

kable(segment_summary, caption = "Customer count by value segment")
```


Thresholds -> Low/Medium: 312.5 Medium/High: 1739 High/Premium: 6087

Table: Customer count by value segment

Segment	Customers	SharePct
Low Value	1085	25
Medium Value	2169	50
High Value	868	20
Premium	217	5

T B I <> ↻ 🖼️ 🔍 ⋮ ⋮ — ψ 😊 ☰ [Close](#)

```
## High-performing vs. underperforming ma
```

To avoid noise from one-off orders, only c
distinct
orders are considered.

High-performing vs. underperforming market

To avoid noise from one-off orders, only
countries with at least 5 distinct orders are
considered.

```
market_perf <- retail_full %>%
  filter(!is.na(Country)) %>%
  group_by(Country) %>%
  summarise(Revenue = sum(Revenue), Orders = n_distinct(InvoiceNo),
            AvgOrderValue = Revenue / Orders, .groups = "drop") %>%
  filter(Orders >= 5) %>%
  arrange(desc(Revenue))

best_market <- market_perf %>% filter(Country != "United Kingdom") %>% slice_t
worst_market <- market_perf %>% arrange(Revenue) %>% slice_head(n = 1)

kable(bind_rows(
  mutate(best_market, Note = "Best (ex-UK)"),
  mutate(worst_market, Note = "Weakest")
), caption = "High-performing vs. underperforming market", digits = 2)
```

Table: High-performing vs. underperforming market

Country	Revenue	Orders	AvgOrderValue	Note
Netherlands	335522	95	3531.8	Best (ex-UK)
Malta	2291	5	458.3	Weakest

Interpretation. The United Kingdom dominates total revenue by a wide margin because the retailer is UK-based, so it is excluded when picking the "high-performing" market for a fairer comparison. Among the remaining markets, `r_best_market$Country` is the strongest, with revenue of £`r_format(round(best_market$Revenue,0), big.mark=",")` across `r_best_market$Orders` orders (average order value £`r_format(round(best_market$AvgOrderValue,2))`) -- consistent with an established export/wholesale relationship. `r_worst_market$Country`, by contrast, is the weakest market meeting the minimum order threshold, generating only £`r_format(round(worst_market$Revenue,0), big.mark=",")` across `r_worst_market$Orders` orders, suggesting little to no active marketing or distribution presence there yet.

\newpage

✓ Task 4: Store and Retrieve Data Using SQL

```
dir.create("output", showWarnings = FALSE)
db_path <- "output/retail_sales.db"
if (file.exists(db_path)) file.remove(db_path)

con <- dbConnect(RSQLite::SQLite(), db_path)

retail_sales_export <- retail_full %>%
  mutate(InvoiceDate = as.character(InvoiceDate))

dbWriteTable(con, "retail_sales", retail_sales_export, overwrite = TRUE)
cat("Rows written to 'retail_sales' table:", dbGetQuery(con, "SELECT COUNT(*) A

Rows written to 'retail_sales' table: 525944
```

✓ Query 1: Top 5 customers by revenue

```
q1 <- dbGetQuery(con, "
  SELECT CustomerID, Country, ROUND(SUM(Revenue), 2) AS TotalRevenue
  FROM retail_sales
  WHERE CustomerID IS NOT NULL
  GROUP BY CustomerID, Country
  ORDER BY TotalRevenue DESC
  LIMIT 5;
")
kable(q1, caption = "SQL: Top 5 customers by revenue")
```

Table: SQL: Top 5 customers by revenue

CustomerID	Country	TotalRevenue
18102	United Kingdom	403365
14646	Netherlands	330057
17450	United Kingdom	181054
16446	United Kingdom	168472
14911	EIRE	152203

✓ Query 2: Total revenue by country

```
q2 <- dbGetQuery(con, "
  SELECT Country, ROUND(SUM(Revenue), 2) AS TotalRevenue, COUNT(DISTINCT InvoiceID) AS Orders
  FROM retail_sales
  WHERE Country IS NOT NULL
  GROUP BY Country
  ORDER BY TotalRevenue DESC
  LIMIT 10;
")
kable(q2, caption = "SQL: Total revenue by country (top 10)")
```

Table: SQL: Total revenue by country (top 10)

Country	TotalRevenue	Orders
United Kingdom	7996411	16649
Netherlands	335522	95
EIRE	288215	260
Germany	235801	457
France	206020	389
Australia	160830	60
Spain	65607	91
Switzerland	57367	52
Belgium	43038	98
Japan	42257	19

```
dbDisconnect(con)
```

✓ Business Insights

1. A quarter of all sales cannot be tied to a known customer or country. (r

nrow(guest_rows) transaction lines (r

`round(100*nrow(guest_rows)/nrow(retail_full),1)` % of the cleaned dataset), worth £`r format(round(sum(guest_rows$Revenue),0), big.mark=",")`, have no `CustomerID`. This blocks loyalty targeting and country-level attribution for a meaningful share of revenue. Capturing a customer or billing-country reference at checkout, even for guest orders, would close this gap.

2. Revenue is heavily concentrated in a small customer segment. The **Premium** segment (top 5% of customers by spend) accounts for a disproportionate share of total revenue relative to its size -- see the top-5 customer table above, where the leading few customers each contribute multiples of a typical **Low Value** customer's total spend. Retention programs aimed at this thin Premium/High Value layer protect more revenue per customer than broad-based promotions to the Low Value segment.

3. The business is structurally dependent on a single home market. The United Kingdom supplies the large majority of both orders and revenue (Table: "Top 5 countries by revenue"), while most other markets, including `r worst_market$Country`, contribute comparatively little. This concentration is a growth opportunity (international expansion headroom) but also a risk -- any disruption to UK demand or logistics would disproportionately affect total revenue. Markets such as `r best_market$Country` show that demand exists outside the UK and could be a template for expansion.

Session Info

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 22.04.5 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblas-p0.3.20.so;
LAPACK version 3.10.0
```

```
locale:
```

```
[1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C
[3] LC_TIME=en_US.UTF-8      LC_COLLATE=en_US.UTF-8
[5] LC_MONETARY=en_US.UTF-8  LC_MESSAGES=en_US.UTF-8
[7] LC_PAPER=en_US.UTF-8     LC_NAME=C
[9] LC_ADDRESS=C             LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
```

```
time zone: Etc/UTC
```

```
tzcode source: system (glibc)
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] knitr_1.51      writexl_2.0.1  DBI_1.3.0      RSQLite_3.53.3 dplyr_1.2.1
[6] readxl_1.5.0    jsonlite_2.0.0 readr_2.2.0
```

```
loaded via a namespace (and not attached):
```

```
[1] bit_4.6.0      compiler_4.6.1  crayon_1.5.3    tidyselect_1.2.1
```